



RAPPORT DU PROJET DE MACHINE LEARNING

Filière : *Sciences des Données, Big Data & Intelligence Artificielle*

TITRE DU PROJET

Estimation des émissions de CO₂ des véhicules

Réalisé par :

- SOMDA Mwingoum Yanis Fadel
- KOUMA Lassana
- IDDOUCHE Oumayma

Encadré par :

Pr. Sara El ATEIF

TABLE DES MATIÈRES

TABLE DES MATIÈRES

1 Introduction et Analyse Préliminaire

1.1	Problématique et Objectif	
1.2	Collecte de données	
1.3	Analyse Exploratoire des Données (EDA)	
1.3.1	Présentation générale du dataset	
1.3.2	Analyse des valeurs manquantes	
1.3.3	Statistiques descriptives des variables numériques	
1.3.4	Analyse des distributions	
1.3.5	Analyse des corrélations	
1.3.6	Analyse par marque	
1.3.7	Synthèse de l'EDA	

2 Prétraitement des données

2.1	Nettoyage des données	
2.1.1	Gestion des valeurs manquantes	
2.1.2	Autres opérations de nettoyage	
2.2	Création d'une variable temporelle : Age_Vehicule	
2.2.1	Méthodologie	
2.2.2	Vérification	
2.3	Optimisation par suppression et Encodage des variables catégorielles	
2.3.1	Optimisation par suppression	
2.3.2	Encodage des variables catégorielles	
2.4	Conclusion du chapitre	

3 Solution Machine Learning

3.1	Séparation du dataset et préparation des jeux de données
3.2	Choix des modèles
3.3	Entraînement des modèles utilisés
3.4	Résultats des modèles
3.4.1	Modèle XGBoost
3.4.2	Modèle Random Forest
3.4.3	Modèle Decision Tree
3.4.4	Modèle MLP
3.5	Comparaison des modèles
3.6	Interprétation du dashboard
3.7	Conclusion du chapitre

4 Démonstration de l'Application

4.1	Objectif de la démonstration
4.2	Saisie des caractéristiques du véhicule
4.3	Interprétation des résultats
4.4	Résumé de la démonstration

5 Conclusion et Perspectives

5.1	Conclusion
5.2	Perspectives

Dépôt Github du projet

Le code complet du projet d'estimation des émissions de CO₂ incluant la préparation des données, la modélisation et l'interface utilisateur avec Flask, est disponible sur le dépôt GitHub suivant :

https://github.com/lassana-kouma/Prediction_Co2_Projet_Machine_learning

Bibliothèques et technologies utilisées

Cette partie présente les principales bibliothèques et technologies utilisées dans le cadre du projet, couvrant l'ensemble des étapes depuis la collecte et la préparation des données, l'entraînement des modèles de machine learning, jusqu'à la création d'une interface utilisateur et la gestion des résultats.

- **Pandas** : utilisée pour le chargement, la manipulation et la structuration des jeux de données sous forme de DataFrame, facilitant les opérations de transformation et de préparation des données.
- **NumPy** : employée pour les calculs numériques, les opérations vectorisées, le calcul des métriques et la génération de valeurs aléatoires nécessaires lors de certaines transformations des variables.
- **Matplotlib** et **Seaborn** : utilisées pour la visualisation graphique et statistique des données, permettant d'analyser les distributions, tendances et relations entre les variables avant l'entraînement des modèles.
- **Scikit-learn** : utilisée pour le prétraitement des données, la structuration des transformations et l'entraînement des modèles d'apprentissage automatique.
- **XGBoost** : bibliothèque spécialisée en apprentissage automatique, employée pour l'entraînement de modèles performants sur des données tabulaires.
- **Pickle** : utilisée pour la sauvegarde des modèles entraînés, permettant leur réutilisation sans relancer la phase d'entraînement.
- **OS** et **Shutil** : utilisées pour la gestion des fichiers et des répertoires, facilitant l'organisation des modèles et des résultats générés.
- **Flask** : framework Python utilisé pour développer une API permettant de servir les modèles et d'interagir avec l'interface utilisateur.
- **HTML** et **CSS** : utilisés pour la création de l'interface utilisateur, avec HTML pour la structure de page et CSS pour le style et la mise en forme.

Chapitre 1

Introduction et Analyse Préliminaire

1.1 Problématique et Objectif

L'augmentation des émissions de CO₂ provenant du secteur automobile constitue un **enjeu environnemental majeur**. Ces émissions sont directement liées au **réchauffement climatique**, à la pollution de l'air et à la dégradation de la santé publique. Dans ce contexte, il est essentiel de pouvoir **quantifier et prédire les émissions de CO₂** d'un véhicule de manière précise et rapide, notamment pour :

- guider les **constructeurs automobiles** dans la conception de véhicules plus écologiques ;
- aider les **autorités** à établir des normes d'émissions ;
- informer les **consommateurs** sur l'impact environnemental de leurs choix.

Objectif du projet :

L'objectif principal du projet est de développer un modèle de **Machine Learning** capable de prédire la quantité d'émissions de CO₂ (en g/km) d'un véhicule, à partir de variables explicatives techniques telles que :

- le type de moteur (essence, diesel, hybride, etc.) ;
- la cylindrée et la puissance ;
- la consommation de carburant ;
- le poids du véhicule et d'autres caractéristiques pertinentes.

Ce modèle permettra non seulement de **prédire les émissions** de véhicules existants, mais aussi de **fournir des informations exploitables** pour la conception de véhicules moins polluants et pour la sensibilisation des utilisateurs.

1.2 Collecte de données

Pour ce projet, nous avons rassemblé **six jeux de données complémentaires** portant sur les émissions de CO₂ des véhicules, représentant plus de **100 000 observations**. Ces données proviennent de sources diverses et fiables, à savoir :

-
- **Kaggle** : jeux de données publics liés aux émissions automobiles.
 - **ADEME - France** : données environnementales officielles.
 - **Government of Canada - Open Data** : données publiques sur les véhicules.
 - **EPA (Environmental Protection Agency) - USA** : informations techniques et environnementales.
 - **Autres sources** obtenues via *web scraping*.

Nature des données

Les données collectées couvrent différents aspects des véhicules, classés comme suit :

- **Données techniques** : taille du moteur, nombre de cylindres, puissance, poids, etc.
- **Données de consommation** : consommation urbaine, extra-urbaine et mixte.
- **Données administratives** : marque, modèle, catégorie, type de carburant, vitesse de transmission.
- **Données temporelles** : âge du véhicule.

Défis principaux rencontrés

La collecte et la consolidation de ces données ont présenté plusieurs difficultés, notamment :

- **Hétérogénéité des données** : les jeux de données avaient des formats différents et des colonnes non uniformes.
- **Variables manquantes** : certaines variables étaient présentes dans certains jeux mais absentes dans d'autres.
- **Formats numériques variés** : par exemple, l'utilisation de la virgule comme séparateur décimal, problème rencontré dans les données provenant des sites français, contrairement aux sites anglophones qui utilisent le point comme séparateur.
- **Différences d'unités** : la puissance pouvait être exprimée en kW ou en chevaux.

Cette diversité a nécessité un **nettoyage approfondi et un prétraitement rigoureux** afin de rendre les données homogènes et exploitables pour le développement du modèle de Machine Learning.

Processus de fusion et d'harmonisation des données

Après l'analyse individuelle des différents jeux de données, une étape de **fusion et d'harmonisation** a été réalisée afin d'obtenir un **dataset global unique et exploitable** pour le projet. Ce processus a suivi plusieurs étapes clés :

1. **Définition d'un schéma commun** : un *DataFrame* global standard a été défini en spécifiant l'ordre et le nom des colonnes (**FINAL_COLUMNS_ORDER**) à appliquer à tous les jeux de données, garantissant ainsi une structure uniforme pour chaque observation.

-
2. **Renommage, ajout et alignement des colonnes** : les colonnes ont été **renommées** selon des dictionnaires spécifiques (`RENAME_MAP_FRANCE_OLD`, `RENAME_MAP_FRANCE_NEW`, `RENAME_MAP_MARS2014`, `RENAME_MAP_CANADA`) afin d'unifier la terminologie entre les différents fichiers. Les colonnes manquantes ont été **ajoutées et initialisées à NaN** pour respecter le schéma global, tandis que les colonnes redondantes ou inutiles ont été **supprimées** (`drop(columns=...)`). Enfin, toutes les colonnes ont été **réordonnées** conformément au schéma défini.
 3. **Harmonisation des formats numériques et des unités** : les colonnes numériques ont été converties aux types appropriés (`float` ou `int`) et les unités standardisées (par exemple, puissance moteur en kW ou en chevaux, consommation en L/100 km) afin de garantir la cohérence des données à travers les différentes sources.
 4. **Concaténation des datasets** : les fichiers prétraités et harmonisés ont été **fusionnés** à l'aide de `pd.concat()` pour constituer un *DataFrame* global unique. Cette étape a permis de regrouper toutes les observations en un seul jeu de données consolidé.

À l'issue de ce processus, un **dataset global homogène et cohérent** a été obtenu, prêt à être utilisé pour les étapes suivantes de transformation des variables et de modélisation par des algorithmes de *Machine Learning*, tout en conservant les valeurs manquantes (NaN) pour les variables spécifiques absentes dans certains fichiers.

1.3 Analyse Exploratoire des Données (EDA)

Cette section vise à analyser en profondeur le jeu de données initial avant toute étape de nettoyage, d'encodage ou de modélisation. L'objectif est de comprendre la structure des données, d'identifier les valeurs manquantes, d'étudier la distribution des variables, ainsi que les relations existantes entre les caractéristiques explicatives et la variable cible.

1.3.1 Présentation générale du dataset

Le jeu de données étudié contient initialement **92 595 observations** et **17 variables**. Ces variables décrivent des caractéristiques techniques, énergétiques et structurelles de véhicules, ainsi que leur niveau d'émissions de dioxyde de carbone (*Emissions CO2 (g/km)*), qui constitue la variable cible du modèle.

Les variables sont de nature mixte :

- **Variables catégorielles** : Marque, Modèle, Version Détaillée, Catégorie de Véhicule, Type de Carburant, Transmission.
- **Variables numériques** : Cylindrée, Cylindres, Puissance moteur, Puissance fiscale, masses du véhicule et consommations.

L'analyse des types révèle une présence importante de colonnes de type *object*, y compris pour certaines variables censées être numériques, indiquant des incohérences de format dès le chargement du dataset.

1.3.2 Analyse des valeurs manquantes

L'analyse de la répartition des valeurs manquantes met en évidence une forte hétérogénéité entre les variables. Certaines colonnes présentent un taux de complétude quasi total, tandis que d'autres contiennent un volume très important de valeurs manquantes.

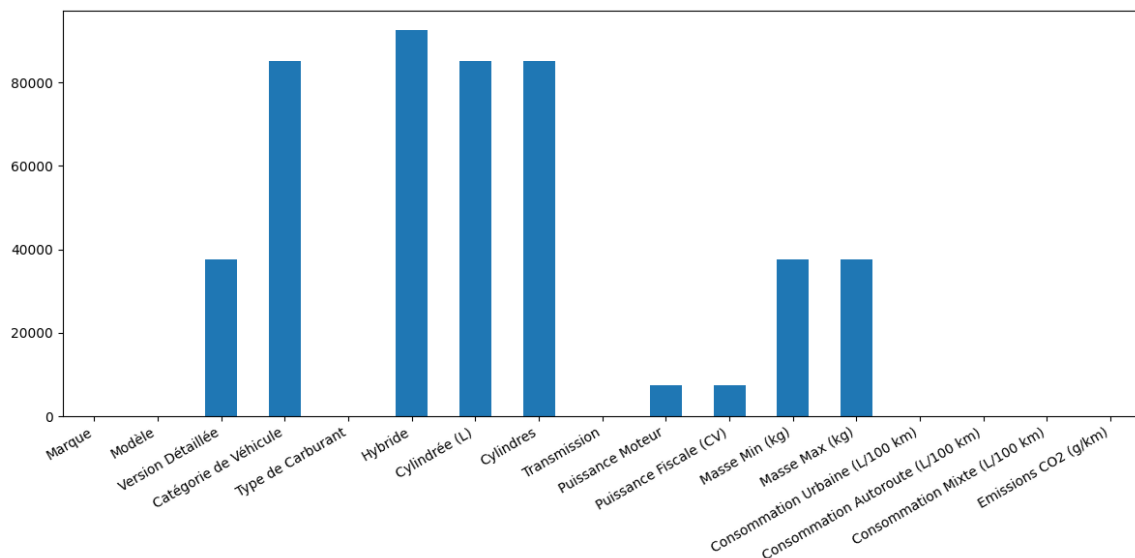


FIGURE 1.1 – Répartition des valeurs manquantes par variable

Les variables les plus concernées sont :

- **Hybride**, est la colonne qui a le plus de valeurs manquantes,
- **Catégorie de Véhicule**, **Cylindrée**, **Cylindres**, sont les colonnes avec le plus de valeurs manquantes après la colonne Hybride.
- **Version Détaillée**, avec près de 40% de valeurs manquantes,
- **Masse Min** et **Masse Max**, avec près de la moitié des observations absentes.

À l'inverse, les variables liées à la consommation et à la variable cible présentent un très faible taux de valeurs manquantes, ce qui garantit une bonne qualité d'information sur les aspects énergétiques des véhicules.

Cette analyse justifie la mise en place de stratégies adaptées de nettoyage, d'imputation ou de suppression de variables dans les étapes suivantes.

1.3.3 Statistiques descriptives des variables numériques

L'étude des statistiques descriptives (moyenne, écart-type, quartiles, minimum et maximum) permet de mieux comprendre l'échelle et la dispersion des données.

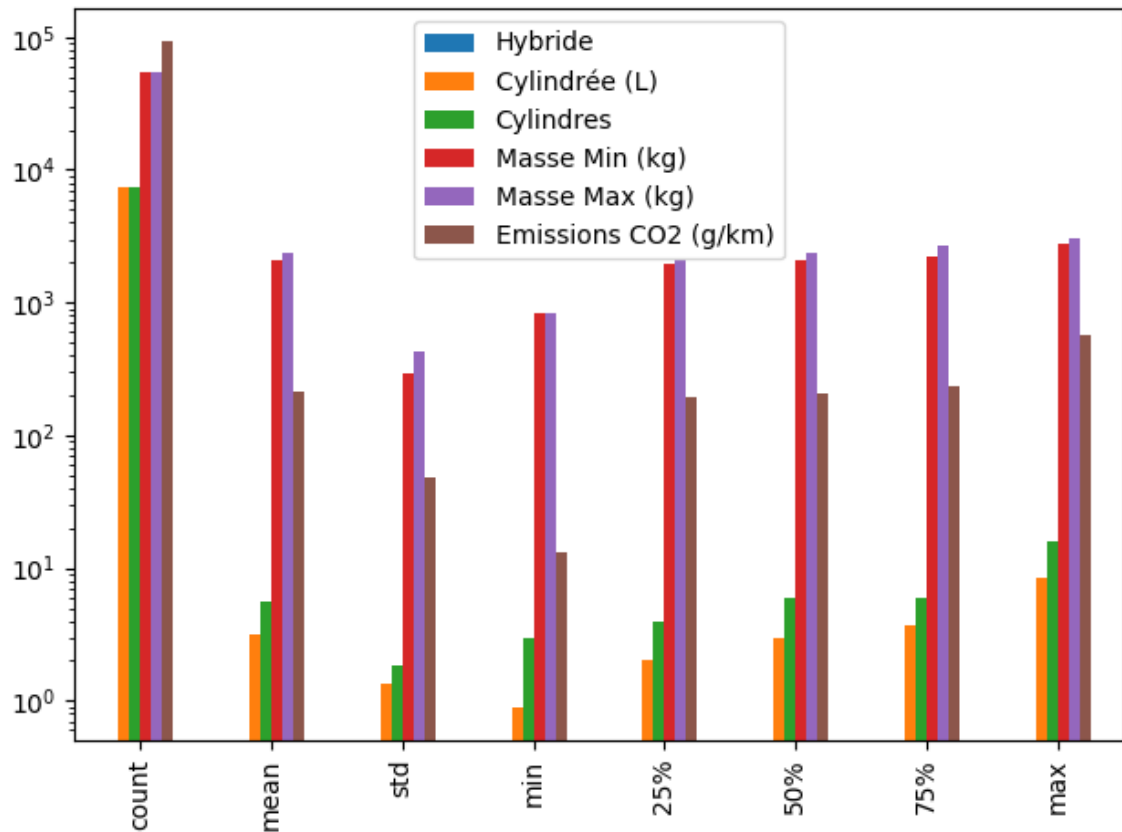


FIGURE 1.2 – Statistiques descriptives des principales variables numériques

Les émissions de CO₂ présentent une moyenne élevée, avec une dispersion importante, traduisant une grande diversité de profils de véhicules. Les consommations (urbaine et autoroute) affichent des distributions étalées et asymétriques, indiquant la présence de valeurs extrêmes.

Les variables liées au moteur (cylindrée, cylindres, puissance) montrent des écarts-types significatifs, confirmant une forte variabilité technologique au sein du domaine automobile étudié.

1.3.4 Analyse des distributions

L'analyse des histogrammes met en évidence des distributions majoritairement asymétriques à droite, particulièrement pour :

- la puissance moteur,
- la puissance fiscale,
- les émissions de CO₂,
- les consommations de carburant.

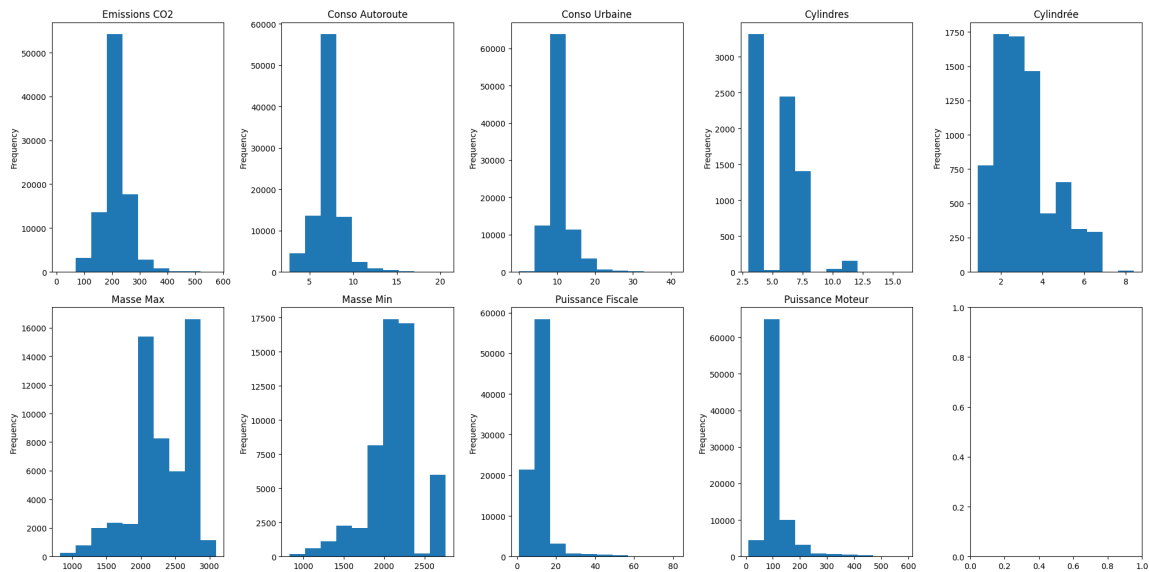


FIGURE 1.3 – Distribution des principales variables numériques

Ces distributions indiquent la présence de valeurs aberrantes correspondant à des véhicules très puissants ou très économes, notamment des modèles sportifs ou de luxe. À l'inverse, la majorité des véhicules se concentre autour de valeurs modérées, ce qui reflète un parc automobile dominé par des véhicules standards.

Cette observation justifie l'usage de modèles capables de gérer les non-linéarités et les outliers, tels que les algorithmes basés sur les arbres.

1.3.5 Analyse des corrélations

La matrice de corrélation révèle des relations fortes entre plusieurs variables explicatives et la variable cible.

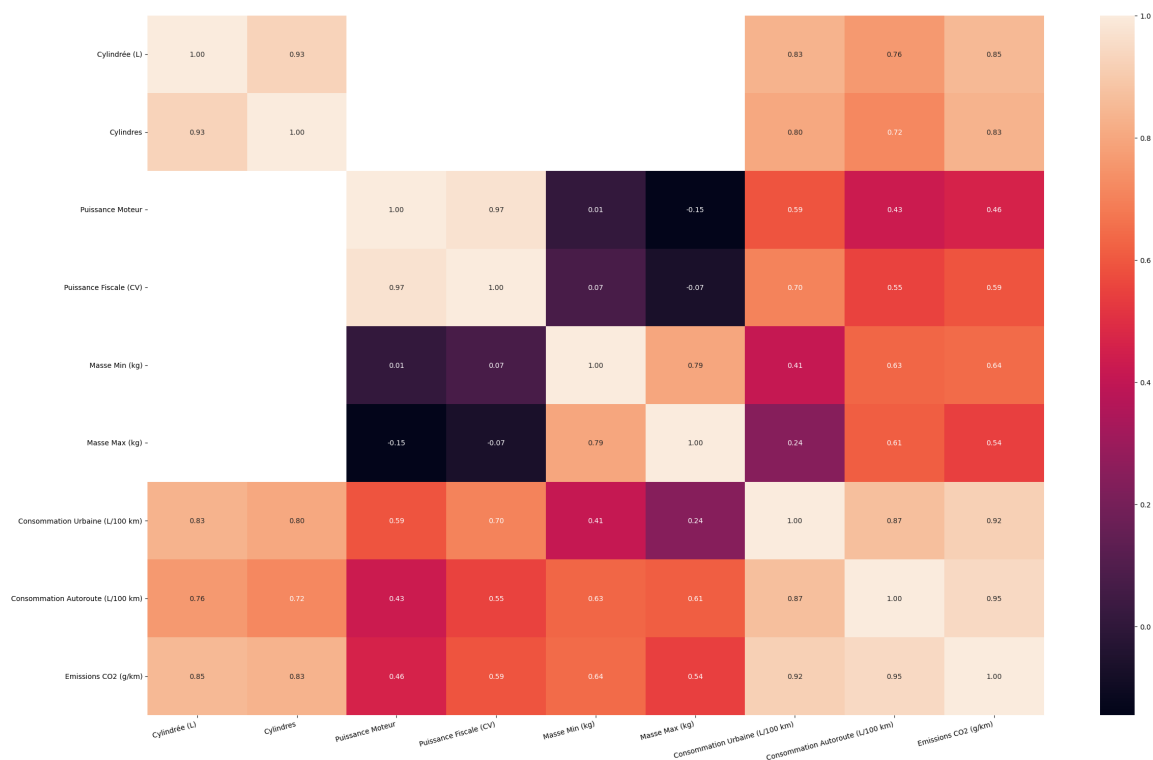


FIGURE 1.4 – Matrice de corrélation entre les variables numériques

Les consommations de carburant présentent une corrélation très élevée avec les émissions de CO₂, notamment :

- Consommation autoroute : corrélation supérieure à 0.9,
- Consommation urbaine : corrélation également très forte.

Les caractéristiques mécaniques, telles que la cylindrée, le nombre de cylindres et la puissance fiscale, sont également fortement corrélées aux émissions, confirmant leur rôle déterminant dans le niveau de pollution d'un véhicule.

Par ailleurs, certaines corrélations très élevées entre variables explicatives (par exemple entre cylindrée et cylindres, ou entre puissance moteur et puissance fiscale) indiquent des redondances potentielles, pouvant mener à des problèmes de multicolinéarité.

1.3.6 Analyse par marque

L'analyse des émissions moyennes par marque met en évidence une forte disparité entre les constructeurs.

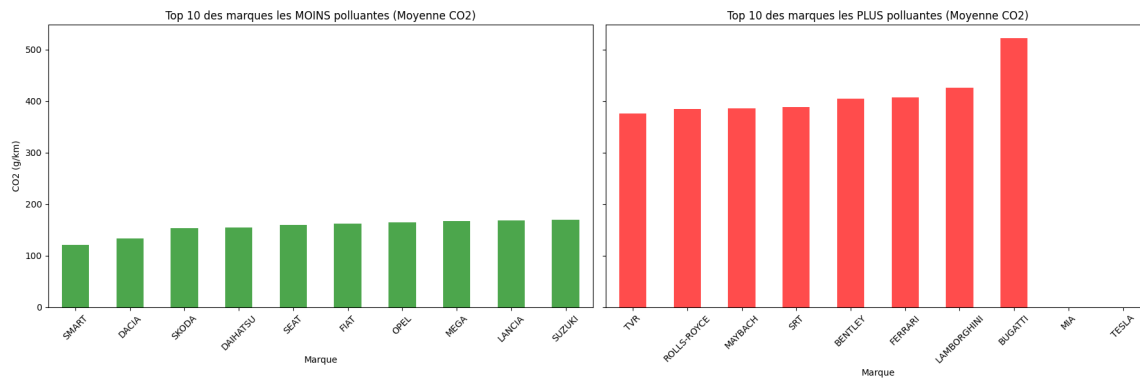


FIGURE 1.5 – Top 10 des marques les moins et les plus polluantes (émissions moyennes de CO₂)

Les marques les moins polluantes sont principalement associées à des véhicules compacts, économiques ou urbains, avec des émissions moyennes relativement faibles. À l'inverse, les marques les plus polluantes correspondent majoritairement à des constructeurs de véhicules de luxe ou sportifs, caractérisés par des moteurs puissants et une consommation élevée.

Cette observation souligne l'importance de la variable *Marque* comme indicateur indirect du positionnement technologique et environnemental des véhicules.

1.3.7 Synthèse de l'EDA

L'ensemble de cette analyse exploratoire a permis :

- d'identifier des problèmes de qualité des données (valeurs manquantes, types incohérents),
- de mettre en évidence des distributions asymétriques et des valeurs extrêmes,
- de révéler des corrélations fortes entre variables explicatives et la cible,
- de justifier les choix méthodologiques ultérieurs en matière de nettoyage, de sélection de variables et de modèles de Machine Learning.

Ces observations constituent une base solide pour les étapes suivantes de prétraitement des données, de feature engineering et de modélisation prédictive.

Chapitre 2

Prétraitement des données

Le prétraitement des données constitue une étape fondamentale dans tout projet de Machine Learning, car la qualité des données conditionne directement la performance et la fiabilité des modèles développés. Dans le cadre de ce projet, les données proviennent de plusieurs sources hétérogènes, ce qui a nécessité un ensemble d'opérations visant à corriger les incohérences, harmoniser les formats et préparer les données pour l'analyse et la modélisation.

2.1 Nettoyage des données

Avant l'entraînement des modèles de Machine Learning, une étape essentielle de nettoyage des données a été réalisée afin de garantir la qualité, la cohérence et la fiabilité du dataset final. Cette phase est d'autant plus importante que la fusion de plusieurs jeux de données issus de différents pays (France, Canada, États-Unis) a introduit des disparités en termes de formats, d'unités de mesure et de complétude des informations.

Le nettoyage des données regroupe plusieurs opérations clés, notamment la gestion des valeurs manquantes, la suppression des lignes dupliquées, la correction des incohérences, ainsi que l'harmonisation des formats numériques et des unités.

2.1.1 Gestion des valeurs manquantes

La fusion de plusieurs sources de données a entraîné la présence d'un nombre significatif de valeurs manquantes dans certaines variables, en particulier pour des caractéristiques qui n'étaient pas disponibles dans tous les jeux de données. Afin de traiter ce problème, des stratégies d'imputation adaptées à la nature des variables ont été mises en place.

Pour les **variables numériques**, l'imputation par la **médiane** a été retenue. Ce choix est motivé par le fait que la médiane est une mesure robuste face aux valeurs aberrantes et permet de préserver la distribution globale des données, contrairement à la moyenne qui peut être fortement influencée par des observations extrêmes. Cette approche contribue ainsi à limiter l'introduction de biais dans le dataset.

Pour les **variables catégorielles**, des méthodes de remplissage séquentiel ont été utilisées, à savoir le **forward fill** et le **backward fill**. Le *forward fill* consiste à remplacer une valeur manquante par la dernière valeur non manquante observée précédemment dans la colonne, tandis que le *backward fill* remplace la valeur manquante par la première valeur non manquante observée après celle-ci. Ces méthodes sont particulièrement adaptées dans un contexte où les données présentent une certaine continuité ou cohérence logique, notamment après l’alignement et le tri des observations lors de la fusion des datasets.

2.1.2 Autres opérations de nettoyage

En complément de la gestion des valeurs manquantes, plusieurs autres opérations de nettoyage ont été réalisées afin d’améliorer la qualité globale des données :

- **Suppression des lignes dupliquées** : certaines observations identiques ont été générées lors de la fusion des différents jeux de données. Ces doublons ont été identifiés et supprimés afin d’éviter une surreprésentation artificielle de certains véhicules.
- **Conversion des types de données** : les colonnes numériques ont été converties vers des formats appropriés (float ou int) afin de garantir leur compatibilité avec les algorithmes de Machine Learning.
- **Normalisation des formats numériques** : les différences de notation entre les sources ont été corrigées, notamment le remplacement de la virgule par le point comme séparateur décimal pour les données issues des sites français, afin d’assurer une lecture correcte des valeurs numériques.
- **Uniformisation des unités de mesure** : certaines variables, comme la puissance moteur, étaient exprimées dans des unités différentes selon les sources (chevaux, kilowatts). Une conversion vers une unité commune a été effectuée pour garantir la cohérence des données.
- **Correction des valeurs incohérentes ou aberrantes** : des valeurs numériques extrêmes, susceptibles de perturber l’apprentissage des modèles, ont été identifiées à l’aide de la méthode de l’intervalle interquartile (IQR). Les quartiles Q1 (1er quartile) et Q3 (3e quartile) ont été calculés, puis l’IQR a été défini comme :

$$IQR = Q3 - Q1$$

Les bornes de détection des outliers ont été établies selon la règle classique :

$$\text{Borne inférieure} = Q1 - 1.5 \times IQR$$

$$\text{Borne supérieure} = Q3 + 1.5 \times IQR$$

Les valeurs situées en dehors de ces bornes ont été plafonnées (*capping*) :

$$x_{\text{traité}} = \begin{cases} \text{Borne inférieure} & \text{si } x < \text{Borne inférieure} \\ \text{Borne supérieure} & \text{si } x > \text{Borne supérieure} \\ x & \text{sinon} \end{cases}$$

Cette approche permet de réduire l'influence des outliers tout en conservant l'intégralité des données, améliorant ainsi la robustesse et la stabilité des modèles de Machine Learning.

- **Nettoyage des chaînes de caractères** : suppression des espaces inutiles ou incorrects au début et à la fin des chaînes de caractères afin d'éviter la création de catégories artificielles distinctes. Retrait des caractères non alphanumériques (excepté les tirets et les espaces).

L'ensemble de ces opérations de nettoyage a permis d'obtenir un dataset homogène, cohérent et fiable, constituant une base solide pour les étapes suivantes d'analyse exploratoire des données, de transformation des variables et de modélisation par des algorithmes de Machine Learning.

2.2 Création d'une variable temporelle : Age_Vehicule

Dans cette section, nous décrivons la création d'une nouvelle variable temporelle, `Age_Vehicule`, à partir des émissions de CO2 des véhicules. L'objectif est de générer une estimation de l'âge du véhicule, tout en introduisant un certain bruit pour réduire la forte corrélation avec les émissions de CO2.

2.2.1 Méthodologie

La création de `Age_Vehicule` se fait en plusieurs étapes :

1. Définition des bornes des émissions de CO2 :

On récupère les valeurs minimale et maximale des émissions de CO2 dans le dataset.

```
co2_min = df['Emissions CO2 (g/km)'].min()
co2_max = df['Emissions CO2 (g/km)'].max()
```

2. Définition de la plage d'âge cible :

On fixe un âge minimum et maximum pour les véhicules.

```
age_min = 1
age_max = 10
```

3. Transformation linéaire :

On applique une transformation linéaire pour convertir les émissions de CO2 en âge approximatif.

```
df['Age_Vehicule'] = (  
    (df['Emissions CO2 (g/km)'] - co2_min) / (co2_max - co2_min)  
    ) * (age_max - age_min) + age_min
```

4. Ajout de bruit aléatoire :

Pour réduire la corrélation directe avec les émissions de CO2, un bruit aléatoire suivant une loi normale (moyenne 0, écart-type 1.5) est ajouté.

```
bruit = np.random.normal(loc=0, scale=1.5, size=len(df))  
df['Age_Vehicule'] = df['Age_Vehicule'] + bruit
```

5. Arrondi :

Enfin, les valeurs sont arrondies pour obtenir des nombres entiers cohérents avec un âge de véhicule.

```
df['Age_Vehicule'] = df['Age_Vehicule'].round().astype(int)
```

2.2.2 Vérification

Pour vérifier la création de la nouvelle variable, on peut visualiser les premières lignes du dataset :

```
df[['Emissions CO2 (g/km)', 'Age_Vehicule']].head()
```

Cette approche permet de disposer d'une variable temporelle `Age_Vehicule` utilisable pour les modèles prédictifs, tout en diminuant l'effet de corrélation directe avec les émissions de CO2.

2.3 Optimisation par suppression et Encodage des variables catégorielles

2.3.1 Optimisation par suppression

Dans le cadre de la préparation des données pour l'entraînement des modèles de Machine Learning, certaines colonnes du dataset ont été jugées **inutilisables ou problématiques** et ont donc été supprimées afin de réduire le bruit, éviter le surapprentissage et garantir la qualité des prédictions.

-
- **Suppression de la colonne *Version Détaillée*** : cette colonne contenait un nombre très élevé de valeurs uniques (3837), ce qui aurait introduit un bruit considérable et un risque important d'*overfitting*. De plus, elle apportait peu d'information supplémentaire par rapport à d'autres colonnes déjà présentes, ce qui la rendait redondante.
 - **Suppression de la colonne *Modèle*** : cette colonne a été supprimée car elle était quasiment similaire à la colonne *Marque*, n'apportant donc pas de valeur ajoutée pour la modélisation..
 - **Suppression de la colonne *Consommation Mixte*** : cette colonne affichait une corrélation très forte avec la variable cible (*Emissions CO2*), ce qui aurait pu permettre au modèle de "tricher" et de se baser directement sur cette feature pour prédire les émissions. Sa suppression a donc été nécessaire pour garantir une modélisation réaliste et robuste.

Cette étape d'optimisation par suppression a permis de réduire la complexité du dataset, de limiter le risque de surapprentissage et de préparer des features plus pertinentes pour l'encodage et la phase d'apprentissage des modèles.

2.3.2 Encodage des variables catégorielles

Après la suppression des colonnes non pertinentes, les variables catégorielles restantes ont été préparées pour l'entraînement des modèles de Machine Learning. Cette étape a consisté en un **nettoyage** et un **encodage** adapté aux algorithmes numériques. Le nettoyage qui avait été fait en amont a permis de réduire le nombre de catégories uniques inutiles et de garantir une meilleure cohérence des données catégorielles.

Identification des colonnes à encoder

Les colonnes sélectionnées pour l'encodage étaient : *Marque*, *Catégorie de Véhicule*, *Type de Carburant*, *Hybride* et *Transmission*. Ces variables présentaient une cardinalité raisonnable et étaient pertinentes pour la prédiction des émissions de CO2. Nous avons donc opté pour le One-Hot Encoding pour l'encodage.

Application du One-Hot Encoding

Pour transformer les variables catégorielles en format numérique exploitable par les modèles, **One-Hot Encoding** a été appliqué :

- Chaque catégorie de chaque variable a été convertie en une colonne binaire (0 ou 1), indiquant la présence ou l'absence de cette catégorie.(0 indiquant l'absence et 1 la présence).
- Les colonnes numériques existantes ont été conservées (`remainder='passthrough'`) pour ne pas perdre les informations quantitatives.

-
- Les valeurs inconnues éventuelles ont été gérées automatiquement grâce à l’option `handle_unknown='ignore'`.

Le résultat est un **DataFrame final encodé**, prêt pour l’entraînement, avec toutes les colonnes catégorielles transformées en variables numériques et les colonnes numériques intactes.

Vérification et sauvegarde du dataset final

Après encodage, plusieurs vérifications ont été réalisées :

- Suppression des doublons pour éviter que des observations identiques n’influencent le modèle.
- Retrait de colonnes techniques superflues (*Unnamed : 0*) générées par la manipulation des fichiers CSV.
- Sauvegarde du dataset final encodé dans un fichier CSV (`dataset_final.csv`), assurant ainsi une base propre et exploitable pour toutes les étapes de modélisation.

Cette étape d’encodage constitue la dernière phase de préparation des données, permettant aux modèles de Machine Learning de traiter correctement l’ensemble des features numériques et catégorielles.

2.4 Conclusion du chapitre

Ce chapitre a été consacré au prétraitement des données, étape essentielle à la réussite de tout projet de Machine Learning. À travers un ensemble d’opérations méthodiques, les données brutes issues de sources hétérogènes ont été nettoyées, harmonisées et structurées afin de garantir leur qualité, leur cohérence et leur exploitabilité.

Les différentes phases de nettoyage ont permis de traiter efficacement les valeurs manquantes, les doublons, les incohérences de formats et d’unités, ainsi que les valeurs aberrantes susceptibles de perturber l’apprentissage des modèles. La création de la variable temporelle *Age_Vehicule* a enrichi le jeu de données en introduisant une information supplémentaire pertinente, tout en limitant une corrélation excessive avec la variable cible. Par ailleurs, l’optimisation par suppression de certaines colonnes redondantes ou trop fortement corrélées a contribué à réduire le bruit et le risque de surapprentissage.

Enfin, l’encodage des variables catégorielles par One-Hot Encoding a permis de transformer l’ensemble des variables en un format numérique compatible avec les algorithmes de Machine Learning, aboutissant à un dataset final propre, homogène et prêt à l’emploi. L’ensemble de ces étapes constitue une base fiable et robuste pour la suite du projet.

Chapitre 3

Solution Machine Learning

3.1 Séparation du dataset et préparation des jeux de données

Avant l'entraînement des modèles, il est essentiel de diviser le dataset en différents ensembles afin d'assurer une évaluation fiable et éviter le surapprentissage. Dans ce projet, nous avons procédé de la manière suivante :

- **Variable cible et variables explicatives** : la variable à prédire est *Emissions CO2 (g/km)*, tandis que les autres colonnes du dataset encodé constituent les variables explicatives.
- **Séparation Train/Test/Validation** :
 - L'ensemble d'entraînement (*train set*) représente les données utilisées pour ajuster les modèles.
 - L'ensemble de test (*test set*) permet d'évaluer la performance finale des modèles sur des données totalement inédites.
 - L'ensemble de validation (*validation set*) est utilisé pour ajuster les hyperparamètres et contrôler le risque de surapprentissage.

Dans notre implémentation, les proportions suivantes ont été adoptées :

- **Entraînement** : environ 66,72% du dataset total
- **Validation** : 20% de l'ensemble d'entraînement donc 16,68% du dataset total
- **Test** : environ 16,6% du dataset total

Cette configuration garantit que chaque modèle est évalué de manière objective et que les hyperparamètres sont optimisés sans fuite d'information.

3.2 Choix des modèles

Pour la prédiction des émissions de CO2, nous avons sélectionné un panel de modèles supervisés de type *regression*, combinant des approches d'arbres décisionnels et de réseaux de neurones. Les modèles choisis sont :

- **Decision Tree (Arbre de décision)** : Un algorithme d'apprentissage supervisé qui frag-

mente le dataset en sous-ensembles de plus en plus petits en fonction de règles de décision (tests sur les variables). C'est un modèle non linéaire capable de capturer des relations complexes entre variables. Il est facile à interpréter et ne nécessite pas de normalisation des variables. Il a été choisi comme baseline (modèle de référence). Sa simplicité permet de mesurer le gain de performance réel apporté par les modèles plus complexes.

- **Random Forest** : ensemble d'arbres de décision construits sur des sous-échantillons aléatoires des données et des variables. Cette approche réduit le surapprentissage et augmente la robustesse en moyennant les prédictions de plusieurs arbres.
- **XGBoost** : algorithme de gradient boosting optimisé, qui construit séquentiellement des arbres en corrigeant les erreurs des arbres précédents. Il a été choisi pour sa grande performance sur les datasets tabulaires et sa capacité à gérer efficacement les relations complexes et non linéaires entre les variables grâce à ses paramètres de régularisation intégrés (`reg_alpha`, `reg_lambda`).
- **MLP (Multi-Layer Perceptron)** : réseau de neurones entièrement connecté capable d'apprendre des relations non linéaires entre les variables. Avec ses couches cachées et sa fonction d'activation *ReLU*, le MLP peut capturer des patterns complexes. Il nécessite toutefois une attention particulière à la normalisation des données et au choix des hyperparamètres. Ce modèle a été retenu pour tester une approche de Deep Learning, permettant d'évaluer si un réseau de neurones peut surpasser les modèles basés sur des arbres pour la prédiction des émissions de CO₂.

Le choix de ces modèles repose sur leur complémentarité : les arbres (Decision Tree, Random Forest, XGBoost) sont très efficaces sur des données tabulaires et gèrent naturellement les relations non linéaires et les interactions entre variables, tandis que le MLP apporte une capacité supplémentaire à modéliser des patterns complexes non capturables par les arbres seuls.

3.3 Entraînement des modèles utilisés

Après la phase de préparation des données et leur encodage, plusieurs modèles de Machine Learning ont été entraînés afin de prédire les émissions de CO₂ des véhicules. L'entraînement s'appuie sur une séparation du dataset en trois ensembles distincts :

- **Ensemble d'entraînement (Train)** : utilisé pour ajuster les paramètres internes des modèles.
- **Ensemble de validation (Validation)** : utilisé pour évaluer les performances intermédiaires et contrôler le surapprentissage.
- **Ensemble de test (Test)** : utilisé uniquement pour l'évaluation finale sur des données jamais vues.

Cette stratégie permet d'assurer une évaluation fiable et robuste des modèles, tout en évitant une optimisation excessive sur un seul sous-ensemble de données.

1. Arbre de décision (Decision Tree)

Le premier modèle entraîné est l'**arbre de décision**, choisi comme modèle de référence (*baseline*). Il s'agit d'un algorithme d'apprentissage supervisé qui segmente récursivement l'espace des données en fonction de règles de décision basées sur les variables explicatives.

Les principaux hyperparamètres utilisés visent à limiter la complexité du modèle et à réduire le risque de surapprentissage :

- `max_depth=11` : limite la profondeur maximale de l'arbre à 11.
- `max_leaf_nodes=65` : limite le nombre de feuilles à 65.
- `min_samples_split=100` et `min_samples_leaf=50` : imposent un nombre minimal d'observations pour effectuer une séparation.
- `criterion='friedman_mse'` : critère de minimisation de l'erreur quadratique moyenne.

Ce modèle présente l'avantage d'être facilement interprétable et de ne nécessiter aucune normalisation des données. Il permet de fournir un point de comparaison clair pour évaluer les gains apportés par des modèles plus complexes.

2. Random Forest

Le deuxième modèle est la **Random Forest**, une méthode d'ensemble basée sur la construction de plusieurs arbres de décision indépendants. Chaque arbre est entraîné sur un sous-échantillon aléatoire des données et un sous-ensemble aléatoire des variables, puis les prédictions sont moyennées.

Cette approche permet de :

- réduire la variance du modèle,
- améliorer la robustesse,
- limiter le surapprentissage par rapport à un arbre unique.

Les principaux hyperparamètres utilisés sont :

- `n_estimators=500` : nombre total d'arbres.
- `max_features='sqrt'` : nombre de variables considérées à chaque séparation.
- `min_samples_split=4` et `min_samples_leaf=2` : contrôle de la taille minimale des nœuds.
- `bootstrap=True` : échantillonnage avec remise pour chaque arbre.

Le modèle est entraîné sur l'ensemble d'apprentissage, puis évalué sur les ensembles de validation et de test afin d'analyser sa capacité de généralisation.

3. XGBoost

Le troisième modèle est **XGBoost** (*Extreme Gradient Boosting*), un algorithme de boosting qui construit les arbres de manière séquentielle. Chaque nouvel arbre cherche à corriger les erreurs commises par les arbres précédents en optimisant une fonction de perte globale.

Les hyperparamètres choisis reflètent un compromis entre performance et régularisation :

- `n_estimators=500` : nombre d'arbres.
- `learning_rate=0.05` : taux d'apprentissage contrôlant la contribution de chaque arbre.

- `max_depth=8` : profondeur maximale des arbres.
- `subsample=0.8` et `colsample_bytree=0.8` : sous-échantillonnage des lignes et des colonnes.
- `reg_alpha=0.1` et `reg_lambda=1` : régularisation L1 et L2 pour limiter l'overfitting.

XGBoost est particulièrement performant sur les données tabulaires et capable de capturer des relations complexes et non linéaires entre les variables explicatives et la variable cible.

4. MLP (Multi-Layer Perceptron)

Enfin, le dernier modèle entraîné est un **MLP**, représentant une approche de Deep Learning. Il s'agit d'un réseau de neurones entièrement connecté composé de plusieurs couches cachées. Dans ce projet, un réseau relativement léger a été utilisé :

- `hidden_layer_sizes=(32, 16)` : deux couches cachées avec respectivement 32 et 16 neurones.
- `activation='relu'` : fonction d'activation non linéaire.
- `solver='adam'` : algorithme d'optimisation adaptatif.
- `early_stopping=True` : arrêt automatique de l'entraînement en cas de stagnation des performances.
- `learning_rate_init=0.001` : taux d'apprentissage initial.

Le MLP a été entraîné sur l'ensemble d'apprentissage et évalué sur les ensembles de validation et de test. Ce modèle permet d'évaluer si une approche neuronale peut surpasser les modèles basés sur des arbres dans la prédiction des émissions de CO₂.

Pour chacun des modèles, plusieurs métriques de performance (MSE, RMSE, MAE et R^2) ont été calculées sur les ensembles d'entraînement, de validation et de test afin d'évaluer la qualité de l'apprentissage, la capacité de généralisation et le risque de surapprentissage.

3.4 Résultats des modèles

3.4.1 Modèle XGBoost

TABLE 3.1 – Performances du modèle XGBoost

Métrique	Entraînement	Test	Validation
MSE	1.953476	14.484346	6.472137
RMSE	1.397668	3.805831	2.544039
MAE	1.014017	1.649261	1.495198
R^2	0.999505	0.996189	0.998314

Le modèle XGBoost offre d'excellentes performances, avec des erreurs faibles et un coefficient de détermination très proche de 1, indiquant une forte capacité de généralisation.

3.4.2 Modèle Random Forest

TABLE 3.2 – Performances du modèle Random Forest

Métrique	Entraînement	Test	Validation
MSE	41.985219	59.927690	61.523784
RMSE	6.479600	7.741298	7.843710
MAE	3.941177	4.625895	4.684207
R ²	0.989360	0.984232	0.983975

Le modèle Random Forest présente des performances satisfaisantes, légèrement inférieures à celles de XGBoost, mais conserve une bonne capacité de généralisation.

3.4.3 Modèle Decision Tree

TABLE 3.3 – Performances du modèle Decision Tree

Métrique	Entraînement	Test	Validation
MSE	123.090402	150.067544	113.233513
RMSE	11.094611	12.250206	10.641124
MAE	6.697855	6.915347	6.751206
R ²	0.968807	0.960514	0.970506

Le modèle Decision Tree affiche des performances plus limitées par rapport aux autres algorithmes, avec des erreurs plus élevées sur les ensembles de test et de validation, indiquant une sensibilité accrue au surapprentissage.

3.4.4 Modèle MLP

TABLE 3.4 – Performances du modèle MLP

Métrique	Entraînement	Test	Validation
MSE	51.324280	56.334942	38.346434
RMSE	7.164097	7.505661	6.192450
MAE	3.796159	3.924296	3.616577
R ²	0.986994	0.985177	0.990012

Le modèle MLP obtient des performances correctes mais reste moins précis que les modèles basés sur les méthodes d'ensemble.

3.5 Comparaison des modèles

Afin d'identifier le modèle le plus performant, les algorithmes ont été comparés sur la base des métriques RMSE, MAE et R^2 calculées sur le jeu de test.

Les résultats montrent que le modèle XGBoost surpasse les autres modèles en termes de précision, suivi par Random Forest, puis le modèle MLP. Le modèle Decision Tree présente les performances les plus faibles.

3.6 Interprétation du dashboard

La Figure 3.1 illustre un tableau de bord interactif développé à l'aide de la bibliothèque **Dash**, permettant de comparer les performances des différents modèles selon la métrique **RMSE** sur l'ensemble de test.

Les valeurs obtenues pour le **RMSE Test** sont les suivantes :

- XGBoost : 3.81 g/km
- Random Forest : 7.74 g/km
- Decision Tree : 12.25 g/km
- MLP (Neural) : 7.51 g/km

Cette visualisation confirme que le modèle **XGBoost** est le plus performant, avec l'erreur quadratique moyenne la plus faible sur l'ensemble de test.

En complément, le dashboard fournit un tableau complet des performances des modèles selon plusieurs métriques (**MSE, RMSE, MAE, R^2**) pour les ensembles d'entraînement, de test et de validation, ce qui permet une comparaison détaillée et une meilleure compréhension de la robustesse des modèles.



FIGURE 3.1 – Comparaison des modèles selon la RMSE Test et tableau des performances complètes

3.7 Conclusion du chapitre

Ce chapitre a présenté la solution de Machine Learning mise en place pour l'estimation des émissions de CO₂ des véhicules. Les résultats obtenus démontrent l'efficacité des modèles

proposés, en particulier XGBoost, qui a été retenu comme modèle final pour la prédiction.

Chapitre 4

Démonstration de l'Application

4.1 Objectif de la démonstration

Cette démonstration vise à illustrer le fonctionnement de l'application **EcoPredict AI**, développée pour estimer les émissions de CO₂ des véhicules à partir de leurs caractéristiques techniques et énergétiques. L'application repose sur une API moderne (**FastAPI**) permettant de fournir des prédictions rapides via une interface web intuitive.

4.2 Saisie des caractéristiques du véhicule

La première étape consiste à remplir le formulaire de saisie du véhicule. La figure 4.1 montre cette interface. L'utilisateur peut renseigner :

- **Identification et moteur** : marque, type de carburant, transmission, statut hybride, cylindrée, nombre de cylindres, puissance moteur et puissance fiscale.
- **Physique et consommation** : masse à vide minimale et maximale, âge du véhicule, consommation urbaine et sur autoroute.

The image shows a web form titled "EcoPredict AI" with the subtitle "Analyse d'impact CO2 par Ensemble Learning". The form is divided into two main sections: "Identification & Moteur" and "Physique & Consommation".

Identification & Moteur

- Marque: Choisir... (dropdown)
- Carburant: Énergie... (dropdown)
- Transmission: Type... (dropdown)
- Hybride ? : Non (dropdown)
- Cylindrée (L): Ex: 2.0 (text input)
- Nb Cylindres: Ex: 4 (text input)
- Puissance Moteur (ch): Ex: 150 (text input)
- Puissance Fiscale (CV): Ex: 8 (text input)

Physique & Consommation

- Masse à vide Min (kg): Ex: 1400 (text input)
- Masse Max (kg): Ex: 1900 (text input)
- Âge (ans): Ex: 2 (text input)
- Conso Urbaine (L/100km): Ex: 8.5 (text input)
- Conso Autoroute (L/100km): Ex: 6.2 (text input)

At the bottom of the form is a large blue button labeled "Calculer l'Empreinte CO2 →".

FIGURE 4.1 – Formulaire de saisie des caractéristiques du véhicule

Cette étape permet de fournir toutes les informations nécessaires aux modèles de prédiction pour estimer l’empreinte carbone du véhicule.

4.3 Interprétation des résultats

Après soumission du formulaire, l’application calcule les prédictions des émissions de CO₂ à l’aide de trois modèles différents : **Random Forest**, **XGBoost** et **MLP (Neural Net)**. La figure 4.2 illustre l’affichage des résultats.

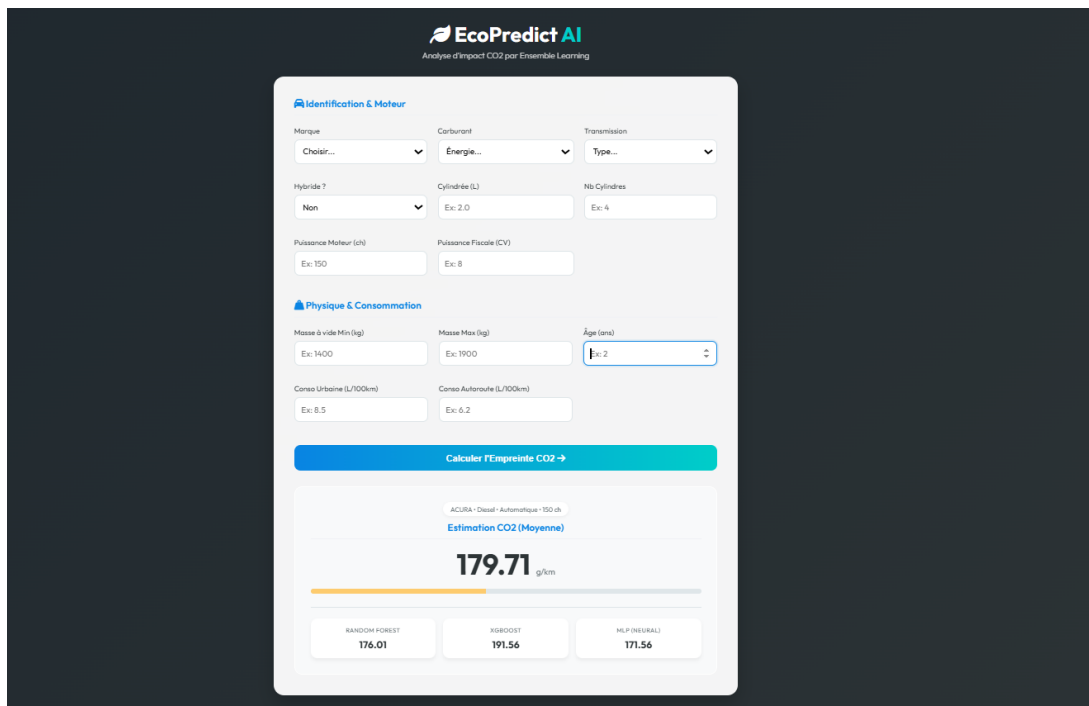


FIGURE 4.2 – Résultats des prédictions des émissions de CO₂ et interprétation graphique

L'interface présente plusieurs éléments clés :

- **Contexte du véhicule** : affiche la marque, le type de carburant, la transmission et la puissance moteur.
- **Estimation CO₂ moyenne** : valeur moyenne des prédictions des trois modèles, exprimée en g/km.
- **Prédictions individuelles par modèle** : chaque modèle fournit sa propre estimation, permettant une comparaison.
- **Barre colorée** : le trait de couleur indique l'intensité de l'émission :
 - **Vert** : émissions faibles (< 120 g/km),
 - **Jaune** : émissions modérées (120–180 g/km),
 - **Rouge** : émissions élevées (> 180 g/km).

Exemple d'interprétation pour un véhicule **ACURA, Diesel, Automatique, 150 ch** :

- Estimation CO₂ moyenne : 179.71 g/km (jaune, émission modérée)
- Random Forest : 176.01 g/km
- XGBoost : 191.56 g/km (rouge, émission élevée)
- MLP (Neural) : 171.56 g/km

Cette présentation permet à l'utilisateur de comprendre immédiatement l'impact environnemental de son véhicule et de comparer les résultats fournis par différents modèles de Machine Learning.

4.4 Résumé de la démonstration

- L'application permet une **saisie simple et intuitive** des caractéristiques d'un véhicule.
- Les prédictions sont effectuées **rapidement grâce à l'API FastAPI**.
- La visualisation colorée facilite l'**interprétation rapide** de l'impact environnemental.
- L'utilisateur peut comparer les estimations de plusieurs modèles pour une meilleure compréhension de la variabilité des prédictions.

Chapitre 5

Conclusion et Perspectives

5.1 Conclusion

Ce projet nous a permis de mettre en pratique les notions théoriques étudiées durant le module de **Machine Learning** au cours de ce semestre, en les appliquant à un cas réel d'estimation des **émissions de CO₂** des véhicules.

Il nous a permis d'acquérir plusieurs compétences techniques et méthodologiques importantes, notamment :

- La mise en œuvre complète d'un **pipeline de machine learning**, depuis le prétraitement des données jusqu'à l'entraînement et l'évaluation des modèles.
- L'utilisation et l'interprétation des **métriques adaptées** aux problèmes de régression, telles que la RMSE, MAE et le coefficient de détermination R^2 .
- La comparaison et l'analyse critique de plusieurs algorithmes de machine learning, permettant de choisir le modèle le plus performant pour le problème étudié.
- La compréhension et la gestion de problématiques comme le **surapprentissage**, le **sous-apprentissage** et la **fuite de données** (*data leakage*).
- L'emploi de techniques d'**optimisation des hyperparamètres** et de **validation croisée** pour améliorer la robustesse et la généralisation des modèles.

5.2 Perspectives

Plusieurs pistes d'amélioration peuvent être envisagées pour enrichir ce travail et rendre l'application plus performante et complète :

- **Application évolutive** : enrichir l'interface de test actuelle en ajoutant des fonctionnalités avancées et des visualisations interactives pour rendre l'utilisation plus complète et conviviale.
- **Modèles plus avancés** : tester des approches basées sur le **deep learning** ou des modèles hybrides combinant plusieurs algorithmes afin d'améliorer la précision des prédictions.

-
- **Évaluation sur de nouvelles données** : tester le modèle sur des données provenant d'autres pays ou d'autres années afin de vérifier sa capacité de généralisation et d'adapter les modèles aux différentes typologies de véhicules.
 - **Intégration de nouvelles variables** : inclure d'autres caractéristiques pertinentes (ex. type de conduite, conditions climatiques) pour affiner les estimations.
 - **Automatisation et API** : déployer l'application sous forme de service web avec API pour permettre un accès facile aux prédictions pour différents utilisateurs et systèmes.

Ainsi, ce projet constitue une base solide pour le développement futur d'outils prédictifs avancés dans le domaine de l'**impact environnemental des véhicules**, avec des applications potentielles pour les constructeurs, les autorités réglementaires et les utilisateurs finaux.